

Network Latency in Distributed LLM Inference

PROGRESS UPDATE: MARCH 4, 2026

Kelsey Penners

About Me!

Kelsey Penners

Master's Student | *Artificial Intelligence & Engineering Systems* | *TU/e*

- Originally from Mountain View, California
- BSc in Computer Science at *University of California, Santa Cruz*
- Got hooked by AI during my bachelor's through a reinforcement learning course
- Outside of research: love discovering new music and increasingly curious about the AI behind music recommendation systems
- Fun Fact: my family's roots are all in the Netherlands and Dutch was my first language, so TU/e felt like home!

My Background

From ML pipelines to distributed LLM inference

- MSc in AI & Engineering Systems, with specialization in High Tech Systems & Robotics – control and systems engineering foundation
- Recent team project: end-to-end deep learning pipeline for defect detection in thermographic data
- Familiar with the general ML development cycle, but this thesis is my first dive into LLMs
- Very interested in this topic due to its relevance in today's world, the clear research gap, and the chance to get hands-on experience with real distributed systems!

Project Overview

How does network communication latency affect distributed LLM inference performance across different hardware interconnects and cluster configurations?

Approach

- LLM serving and benchmarking with vLLM framework and Meta's Llama models
- Hardware: funnotch (4xV100, PCIe) and shy-fec (8xA100, NVLink)
- Three-phase study: single GPU, intra-node, inter-node distribution

Key Metrics

- TTFT – time to first token (prefill sensitivity)
- ITL – inter-token latency (decode sensitivity)
- Throughput – output tokens per second (system capacity)

Prefill is compute-bound, decode is memory-bound; expecting these to respond differently as network latency increases.

Research Phases

How does network communication latency affect distributed LLM inference performance across different hardware interconnects and cluster configurations?

Phase	Scope	Hardware	Research Sub-question
Phase 1	Single GPU baseline	A100, V100	What is the single-GPU performance baseline across sequence lengths and concurrency?
Phase 2	Multi-GPU intra-node	NVLink vs PCIe	How do interconnect types and injected latency affect inference performance?
Phase 3	Heterogeneous inter-node	Cross-node Ethernet	How does Ethernet latency degrade performance across nodes?

Each phase adds a new communication layer, with Phase 1 giving us a reference point.

Phase 1: Experimental Setup

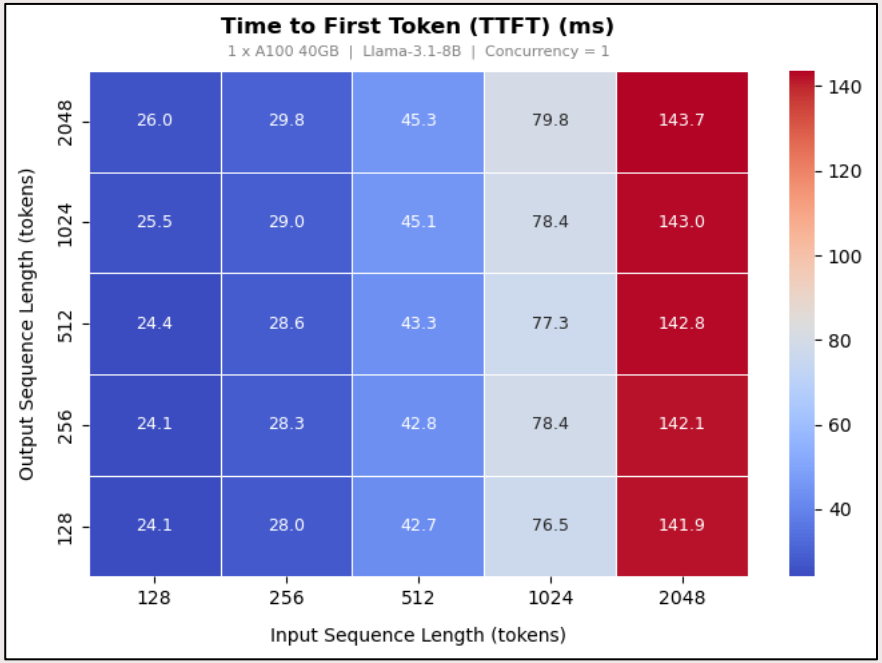
Hardware & Model

- 1 x NVIDIA A100-40GB
- 1 x V100-32GB (in progress)
- *Model*: Llama-3.1-8B, a dense transformer
- *Framework*: vLLM 0.11.2, BFloat16 precision

Experiments

- **Input/output sweep**: 128-2048 tokens (25 total configs)
  - Max concurrency = 1, measuring pure model latency
  - 100 synthetic prompts per config, averaged across ~3 runs
- **Concurrency sweep:** 1-512 concurrent requests
  - Fixed input/output = 512/512 tokens
  - Measures batching behavior and KV cache saturation

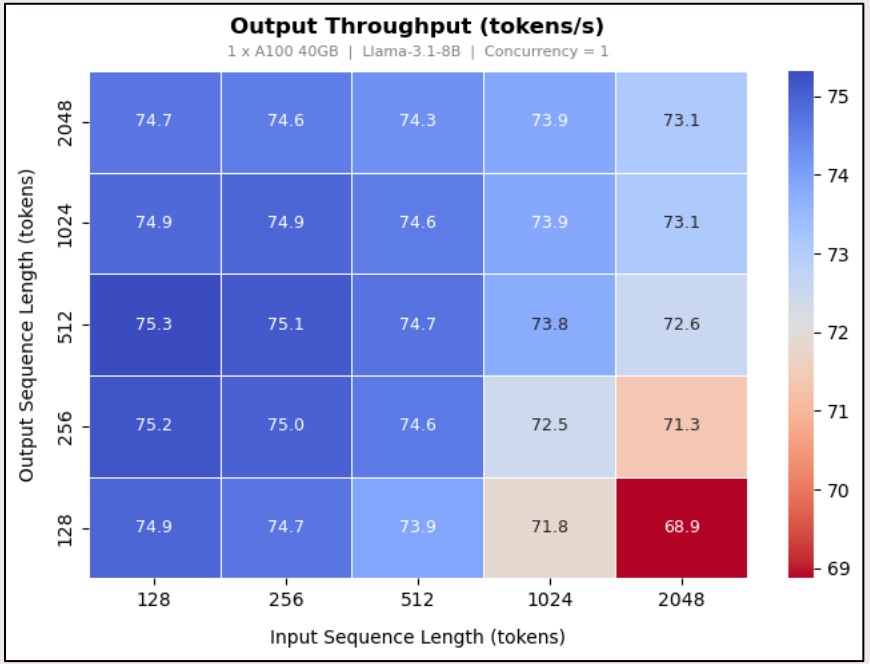
TTFT Scales Linearly with Input Length



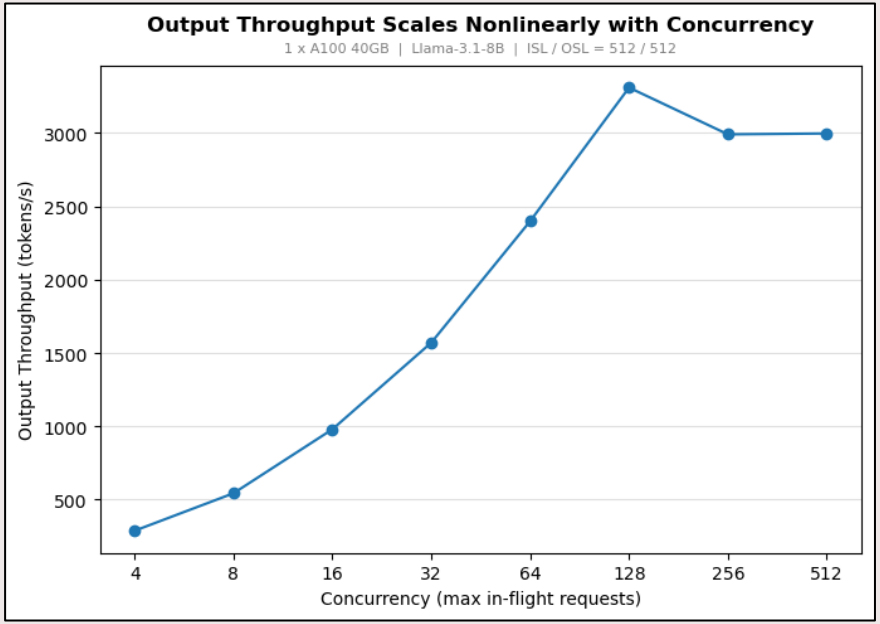
- TTFT almost entirely driven by ISL; this is our prefill metric
- ~2ms spread across all OSL values at any ISL
- Linear relationship: ~0.063 ms/token; our A100 prefill rate baseline
- Small OSL effect likely due to KV cache pre-allocation for longer outputs

Throughput Reflects Decode-Prefill Ratio

- *Lowest at* ISL=2048, OSL=128; most time is spent in prefill, few output tokens are generated
- *Highest at* ISL=128, OSL=512: decode dominated, GPU stays busy generating
- Overall tight range at concurrency=1; throughput variance comes with batching



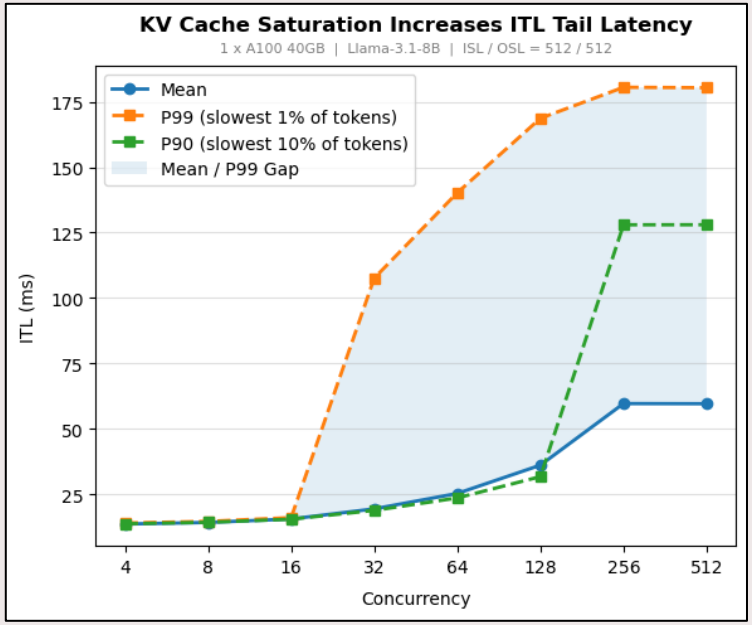
Throughput Plateaus After KV Cache Saturation



- A100-40GB with Llama-3.1-8B loaded (16GB in BF15), about 18GB left for KV cache
- Gain in throughput until concurrency 128, then plateau
- KV cache saturated, requests begin queueing
- Dip after 128 potentially due to scheduling overhead of concurrent requests

ITL Tail Latency Inflation with Concurrency

- Mean ITL stays relatively flat until concurrency 64, plateaus at ~60ms from concurrency 256 on.
- P99 jumps between concurrency 16 and 32: KV cache saturation begins, request queuing affects tail tokens
- P90 jumps between concurrency 128 and 256: further cache saturation
- Shows the system under stress.

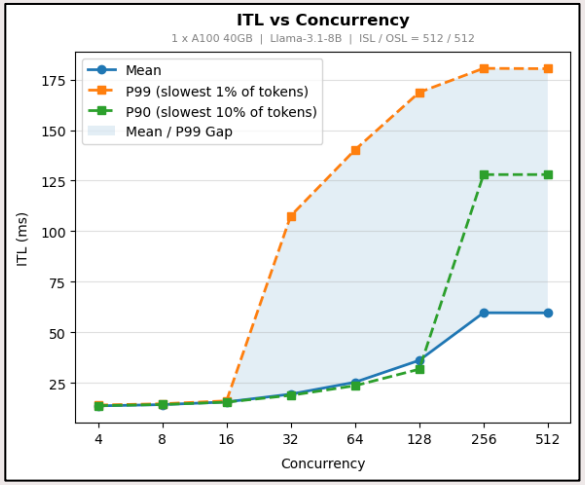


What's Next?

- Finish A100 vs V100 comparison:
  - Same ISL/OSL sweep is running on v100; will show impact of GPU generation on prefill rate and decode throughput
- Move to Phase 2: multi-GPU + models with more parameters
  - Tensor parallelism across GPUs on both servers; will compare PCIe vs NVLink behavior as-is before any injection
- Implement latency injection:
  - Investigating wrapping vLLM's NCCL calls to introduce controlled delays; a software-based approach

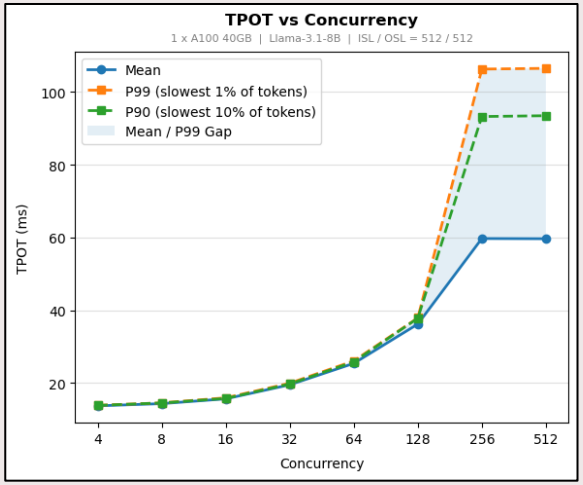
Thank you!

Inter-Token Latency (ITL) vs Time-Per-Output-Token (TPOT)



ITL = fine grained between token latencies

TPOT = averaged time per token across all requests



Run-to-run Variance, Visualized

